



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Práctica:

“Procesador Monociclo RISC-V”

Unidad de aprendizaje:

Arquitectura de Computadoras

Grupo: 5CV2

Integrantes:

- Angela Alessandra Berrospe Negrete
 - Patlan Gualo Luis Eduardo
- Erreguin Franco Yair Alejandro
- Zúñiga Bolaños Diego Samuel

Profesora:

Maria Elena Aguilar Jauregui

Martes 06 de enero del 2026

Índice

Introducción	3
Objetivos	3
Marco teórico.	3
Metodología	5
Resultados.....	6
Conclusiones	13
Bibliografías	14

Introducción

El diseño de un procesador es uno de los temas principales en la arquitectura de computadoras, ya que integra conceptos de lógica digital, organización de hardware y ejecución de instrucciones. Dentro de este contexto, la arquitectura RISC-V se ha convertido en un estándar abierto y moderno, ampliamente adoptado tanto en el ámbito académico como en la industria, debido a su simplicidad, modularidad y extensibilidad. Un procesador monociclo es una arquitectura en la que cada instrucción se ejecuta completamente en un solo ciclo de reloj. Esto implica que todas las etapas del ciclo de instrucción (búsqueda, decodificación, ejecución, acceso a memoria y escritura de resultado) se realizan dentro de un mismo periodo. Aunque este enfoque no es eficiente en términos de rendimiento, resulta ideal para fines educativos, ya que permite comprender claramente el flujo de datos y las señales de control dentro del procesador. En la presente práctica se diseñó e implementó un procesador monociclo basado en el subconjunto RV32I de la arquitectura RISC-V, utilizando el lenguaje de descripción de hardware VHDL. El objetivo principal fue analizar e integrar los distintos bloques funcionales del procesador, así como verificar su correcto funcionamiento mediante simulación.

Objetivos

Objetivo general

Diseñar e implementar un procesador monociclo RISC-V en VHDL, comprendiendo el flujo de datos y las señales de control necesarias para la ejecución de instrucciones básicas del subconjunto RV32I.

Objetivos específicos

- Analizar el conjunto de instrucciones RV32I y sus formatos.
- Diseñar e implementar los bloques principales de un procesador monociclo.
- Implementar el camino de datos (**datapath**) completo en VHDL.
- Diseñar la unidad de control para la generación de señales de control.
- Simular y verificar la correcta ejecución de instrucciones aritméticas, de memoria y de control de flujo.

Marco teórico.

Arquitectura RISC-V (RV32I)

RISC-V es una arquitectura de conjunto de instrucciones (ISA) de tipo RISC, caracterizada por instrucciones de longitud fija de 32 bits, un banco de 32 registros de propósito general y un diseño orientado a la eficiencia del hardware. En esta práctica se implementó el subconjunto RV32I, que incluye instrucciones aritméticas y lógicas tipo R, instrucciones inmediatas tipo I, acceso a memoria tipo Load/Store, así como instrucciones de salto y bifurcación.

Procesador monociclo

En un procesador monociclo, cada instrucción se ejecuta completamente en un único ciclo de reloj. El periodo del reloj debe ser suficientemente largo para soportar la instrucción más lenta. Aunque esta arquitectura presenta limitaciones de rendimiento, su simplicidad facilita el análisis del comportamiento interno del procesador.

Camino de datos (Datapath)

El datapath del procesador monociclo RISC-V implementado está compuesto por los siguientes bloques:

- **Contador de Programa (PC):** almacena la dirección de la instrucción actual y se actualiza con PC+4, una dirección de salto o una dirección de bifurcación.
- **Banco de Registros:** contiene 32 registros de 32 bits, con dos puertos de lectura asíncrona y un puerto de escritura síncrona. El registro x0 permanece siempre en cero.
- **Unidad Aritmético-Lógica (ALU):** realiza operaciones aritméticas y lógicas controladas por la unidad ALU Control.
- **Memoria de Datos:** permite realizar operaciones de lectura y escritura para instrucciones tipo Load y Store.
- **Generador de Inmediatos:** extrae y extiende a 32 bits los campos inmediatos según el formato de la instrucción.
- **Multiplexores:** seleccionan operandos de la ALU, el dato de escritura y el siguiente valor del PC.

Unidad de control

La unidad de control es un bloque de lógica combinacional que decodifica el campo **opcode** de la instrucción y genera las señales **RegWrite**, **ALUSrc**, **MemRead**,

MemWrite, MemtoReg, Branch, Jump y ALUOp, las cuales gobiernan el funcionamiento del datapath.

Metodología

Herramientas y Recursos Utilizados

1. Lenguaje de Descripción de Hardware: **VHDL**
2. Software de simulación y síntesis: **ModelSim / Intel Quartus**
3. Arquitectura objetivo: **Procesador monociclo RISC-V (RV32I)**

Proceso de implementación

1. Contador de Programa (PC)

El PC se implementó como un registro síncrono con señal de reset. Su valor se actualiza en cada flanco de subida del reloj con el valor de next_pc, el cual puede corresponder a PC+4, una dirección de salto o una dirección de bifurcación.

2. Unidad de Control

La unidad de control decodifica el opcode de la instrucción y genera las señales necesarias para controlar el datapath, habilitando o deshabilitando la escritura de registros, el acceso a memoria y la selección de operandos.

3. Banco de Registros

El banco de registros consta de 32 registros de 32 bits. La lectura de rs1 y rs2 es asíncrona, mientras que la escritura en rd es síncrona y controlada por la señal RegWrite. El registro x0 se mantiene siempre en cero conforme a la especificación RISC-V.

4. ALU y ALU Control

La ALU realiza operaciones aritméticas y lógicas sobre operandos de 32 bits. El tipo de operación se define mediante la unidad ALU Control, la cual combina la señal ALUOp con los campos funct3 y funct7 de la instrucción.

5. Generación de inmediatos

Se implementó lógica combinacional para generar los inmediatos de instrucciones tipo I, S, B y J, extendiéndolos a 32 bits según el formato correspondiente.

6. Memoria de Datos

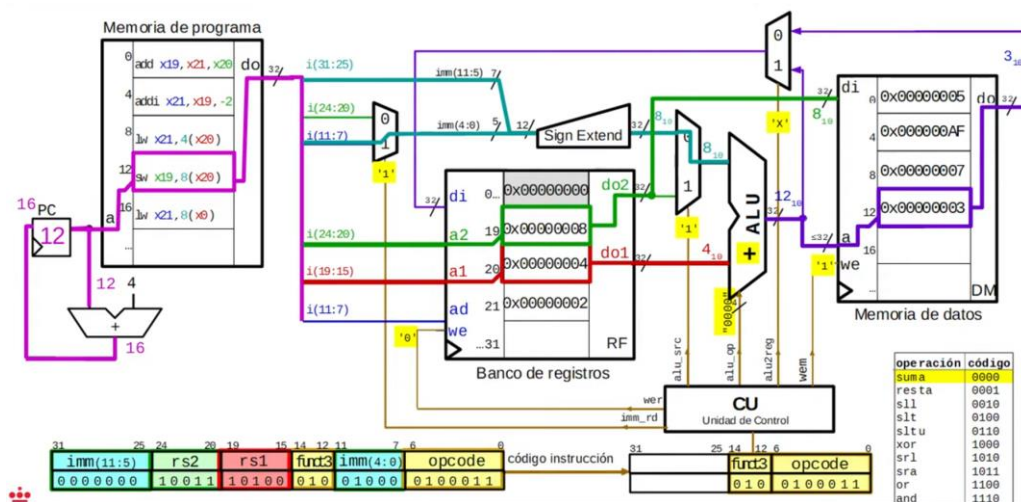
La memoria de datos permite realizar operaciones de lectura y escritura controladas por las señales MemRead y MemWrite. La dirección de acceso proviene del resultado de la ALU.

7. Lógica de actualización del PC

El siguiente valor del PC se selecciona priorizando instrucciones de salto (Jump), bifurcaciones tomadas (Branch y zero_flag) y, en caso contrario, el incremento secuencial PC+4.

8. Simulación y verificación

Se desarrolló un testbench para verificar la correcta ejecución de instrucciones aritméticas, de acceso a memoria y de control de flujo.



Resultados

Durante la simulación se comprobó el funcionamiento correcto del procesador monociclo RISC-V. Entre los resultados obtenidos se observó:

- Ejecución correcta de instrucciones aritméticas tipo R e inmediatas.

- Escritura y lectura adecuada del banco de registros.
- Funcionamiento correcto de las instrucciones de carga y almacenamiento.
- Actualización correcta del PC para instrucciones secuenciales, saltos y bifurcaciones.

Los resultados validan la correcta implementación del datapath y la unidad de control conforme al diseño teórico.

Para el análisis de la simulación se asumen las siguientes condiciones estándar:

- Arquitectura RISC-V monociclo.
- El contador de programa (PC) se incrementa de 4 en 4.
- La escritura en el banco de registros ocurre en el flanco de subida del reloj.
- `reg_data` representa el dato escrito en el registro destino.
- `alu_result` corresponde a la salida de la ALU durante ese ciclo.
- La memoria de datos está inicializada en cero.

Señales observadas

Durante la simulación se analizaron las siguientes señales principales:

- `/tb_risc_v/clk`
- `/tb_risc_v/reset`
- `/tb_risc_v/instruction`
- `/tb_risc_v/pc_out`
- `/tb_risc_v/alu_result`
- `/tb_risc_v/reg_data`

Estas señales permiten verificar el flujo correcto de instrucciones, el avance del PC y la correcta operación de la ALU, registros y memoria.

Análisis por ciclos de reloj

Reset (0 – 20 ns)

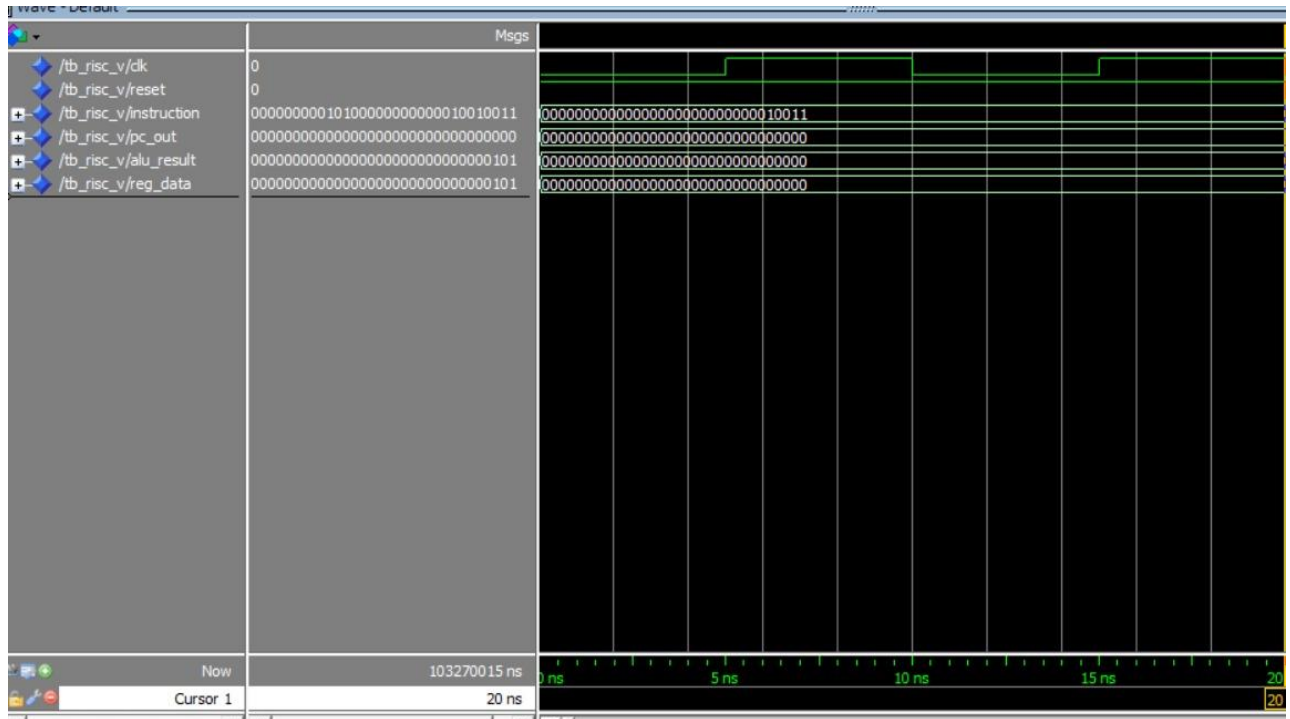
Durante el periodo de reset el procesador no ejecuta ninguna instrucción.

Comportamiento esperado:

- `reset = 1`
- `pc_out = 0x00000000`

- instruction = NOP
- alu_result = 0x00000000
- reg_data = 0x00000000

En esta etapa el PC permanece en cero y no se realiza ninguna operación.



Ciclo 1 (20 – 30 ns)

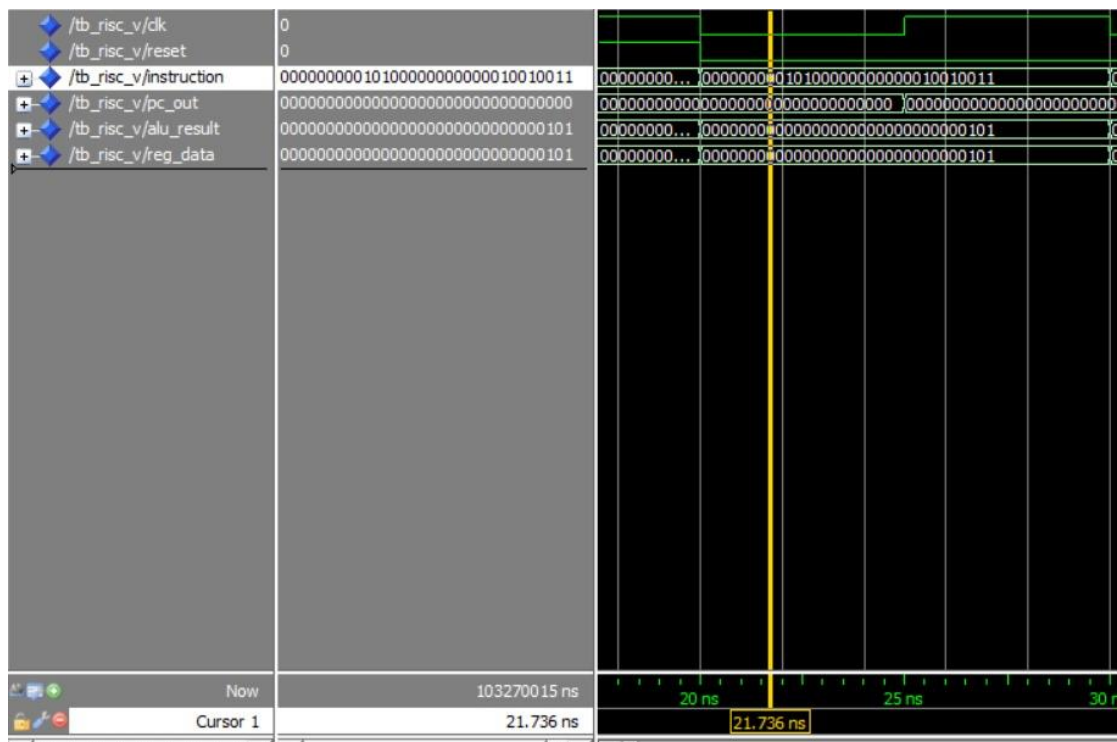
Instrucción: ADDI x1, x0, 5

Descripción: El registro x0 siempre contiene el valor cero, por lo que la ALU suma 0 + 5 y el resultado se escribe en x1.

Señales esperadas:

- pc_out = 0x00000000
- alu_result = 0x00000005
- reg_data = 0x00000005
- instruction = 0x00500093

En el flanco de subida del reloj se realiza la escritura: $x1 \leftarrow 5$.



Ciclo 2 (30 – 40 ns)

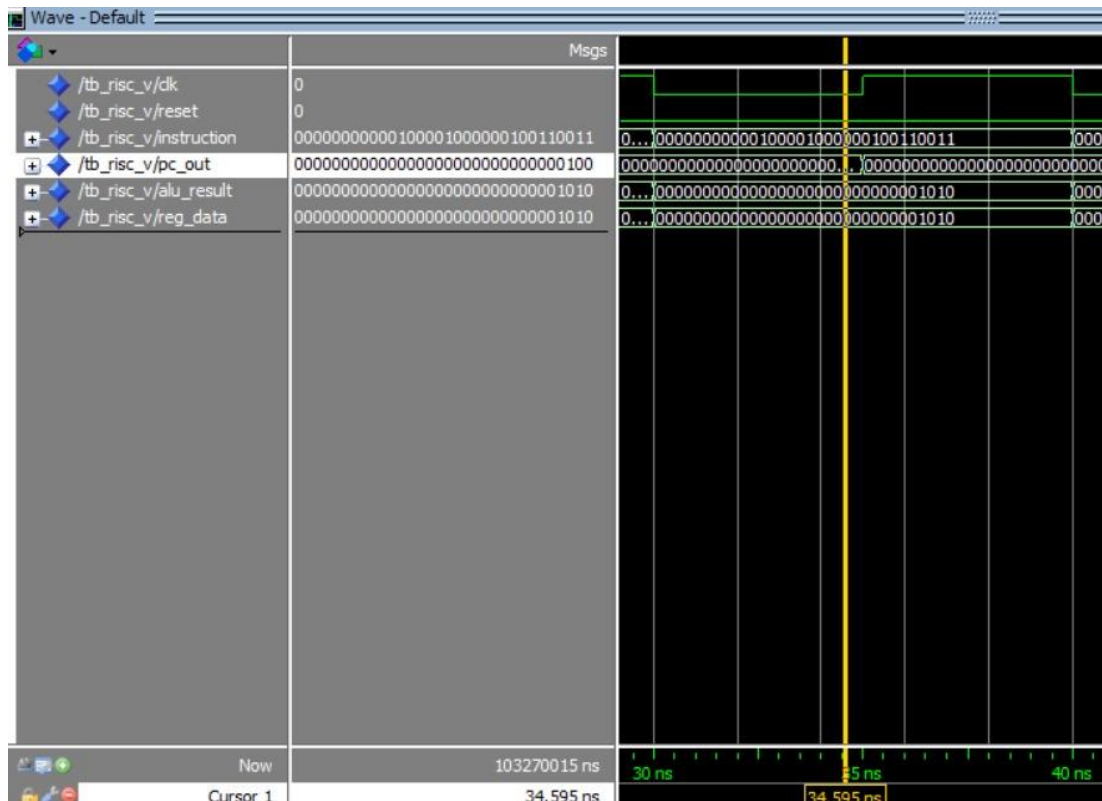
Instrucción: ADD x2, x1, x1

Descripción: La ALU suma el contenido de x1 consigo mismo (5 + 5), obteniendo como resultado 10, el cual se almacena en x2.

Señales esperadas:

- pc_out = 0x00000004
- alu_result = 0x0000000A
- reg_data = 0x0000000A
- instruction = 0x00108133

En el flanco de subida del reloj se realiza la escritura: $x2 \leftarrow 10$.



Ciclo 3 (40 – 50 ns)

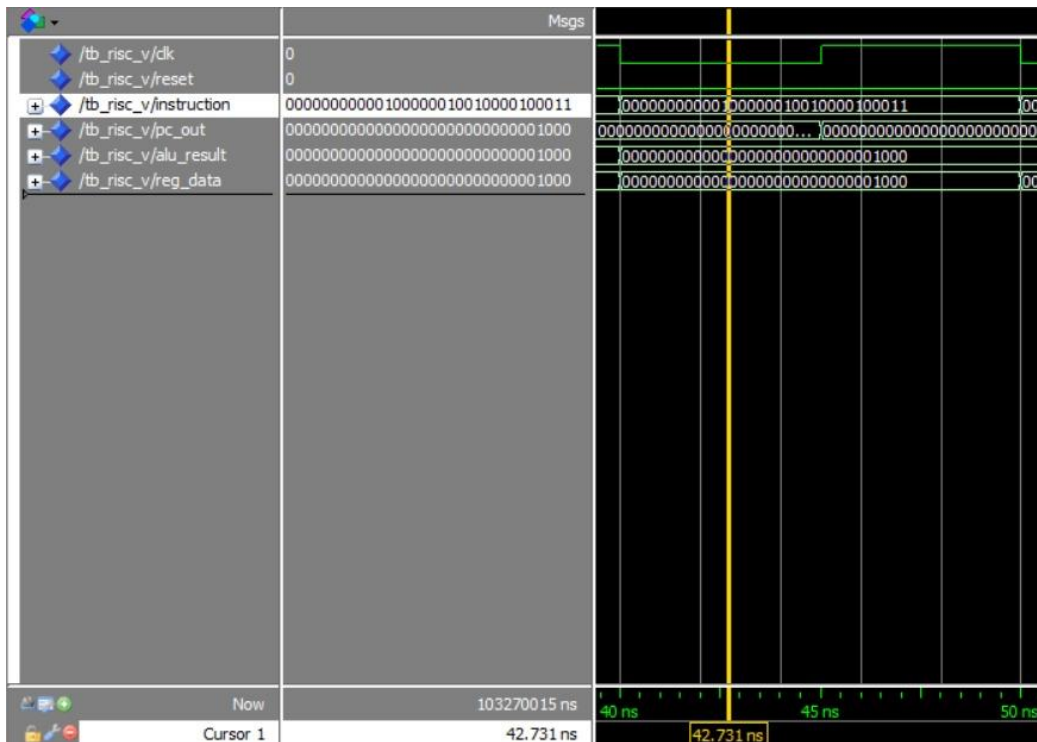
Instrucción: SW x1, 8(x0)

Descripción: La ALU calcula la dirección efectiva sumando $x0 + 8$, obteniendo la dirección 8. El valor de x1 (5) se almacena en la memoria de datos en dicha dirección.

Señales esperadas:

- pc_out = 0x00000008
- alu_result = 0x00000008
- reg_data = 0x00000000 (no utilizado)
- instruction = 0x00102423

En este ciclo no se escribe ningún registro, únicamente se realiza la operación de escritura en memoria.



Ciclo 4 (50 – 60 ns)

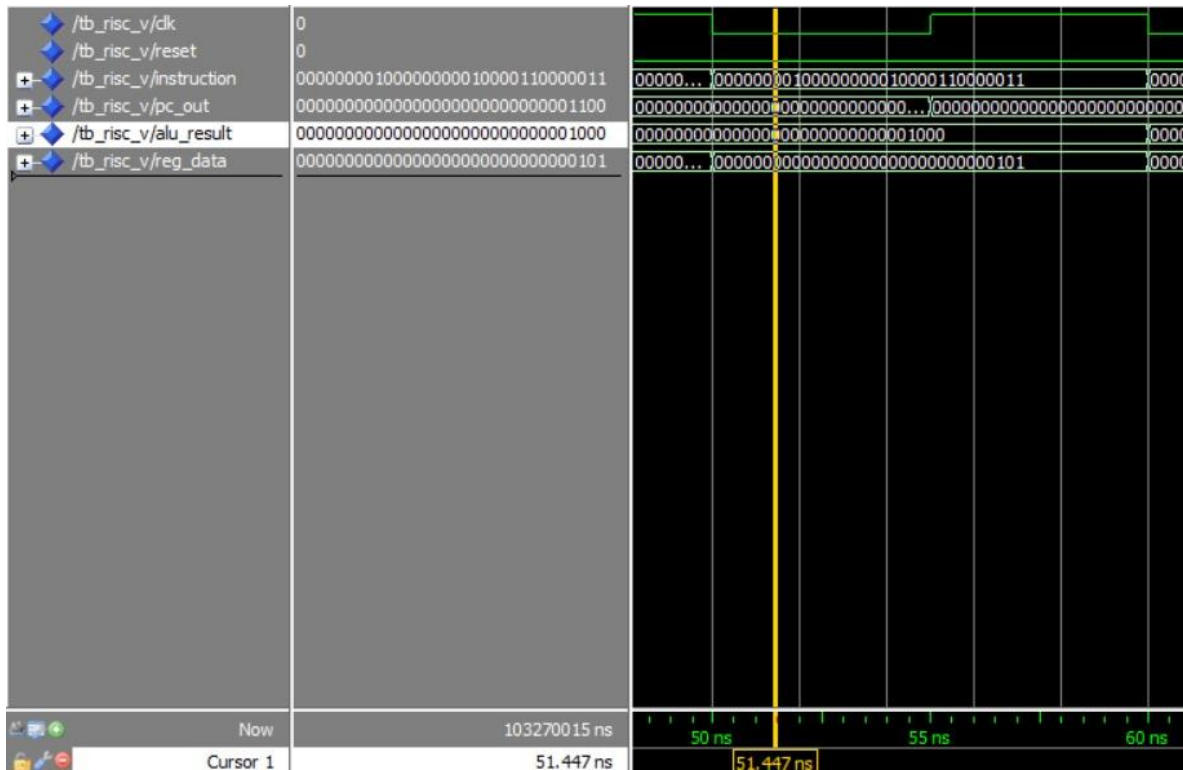
Instrucción: LW x3, 8(x0)

Descripción: La ALU calcula nuevamente la dirección efectiva 8. El procesador lee el contenido de la memoria en esa dirección (valor 5) y lo escribe en el registro x3.

Señales esperadas:

- pc_out = 0x0000000C
- alu_result = 0x00000008
- reg_data = 0x00000005
- instruction = 0x00802183

En el flanco de subida del reloj se realiza la escritura: $x3 \leftarrow 5$.



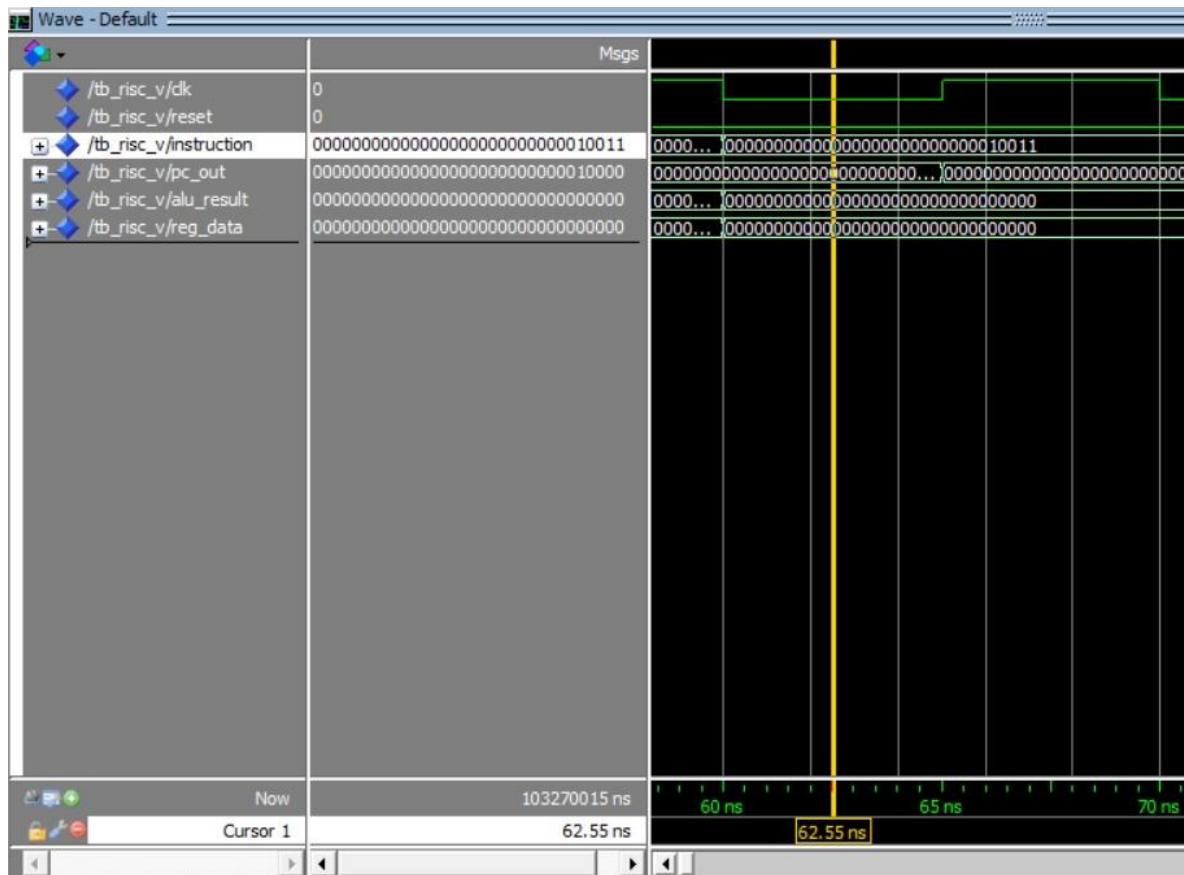
Ciclo 5 (60 – 70 ns)

Instrucción: NOP (equivalente a ADDI x0, x0, 0)

Descripción: No se realiza ninguna operación efectiva ni se modifica el estado del procesador.

Señales esperadas:

- pc_out = 0x00000010
- alu_result = 0x00000000
- reg_data = 0x00000000
- instruction = 0x00000013



Al concluir la simulación, el estado del procesador y la memoria es el siguiente:

Banco de registros:

x1 = 5 x2 = 10 x3 = 5 mem[8] = 5

Conclusiones

La implementación de un procesador monociclo RISC-V permitió entender el funcionamiento interno de un procesador, así como la interacción entre el flujo de datos y las señales de control. Aunque esta arquitectura no es eficiente en términos de rendimiento, resulta fundamental como base para arquitecturas más avanzadas.

Erreguin Franco Yair Alejandro:

La práctica permitió comprender de manera completa el ciclo de instrucción de un procesador, desde la obtención de la instrucción mediante el PC hasta la escritura

del resultado en el banco de registros. Al tratarse de una arquitectura monociclo, fue posible observar cómo todas las etapas se ejecutan dentro de un solo ciclo de reloj, lo que facilita el análisis del flujo de datos y de las dependencias entre módulos. También permitió observar la importancia de la Unidad de Control la cual genera las señales necesarias para controlar el flujo de datos a partir del OpCode.

Patlan Gualo Luis Eduardo:

Se comprobó que la unidad de control es el elemento central que coordina el funcionamiento del procesador. A partir del opcode, esta unidad genera señales críticas como RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch y Jump, las cuales determinan el comportamiento de cada bloque del datapath. Un error en estas señales impacta directamente en la ejecución correcta de las instrucciones.

Angela Alessandra Berrospe Negrete:

La lógica de actualización del PC mostró cómo se implementa el **control de flujo** a nivel hardware. El uso de la señal zero_flag de la ALU junto con Branch y Jump permitió implementar correctamente bifurcaciones condicionales y saltos incondicionales, evidenciando cómo decisiones lógicas simples impactan directamente en el flujo de ejecución del programa.

Zúñiga Bolaños Diego Samuel

La implementación en VHDL permitió trasladar conceptos teóricos a un diseño funcional, reforzando el entendimiento de cómo un procesador pasa de ser un diagrama conceptual a un sistema digital implementable. Además, la simulación validó la correcta integración de los módulos y el cumplimiento del comportamiento esperado.

Bibliografías

- Patterson, D. A., & Hennessy, J. L. (2018). *Computer Organization and Design MIPS Edition: Hardware/Software Interface*. Morgan Kaufmann.
- Harris, S., & Harris, D. (2016). *Digital Design and Computer Architecture*. Morgan Kaufmann Publishers.
- Waterman, A., & Asanović, K. (2019). *The RISC-V Instruction Set Manual*. RISC-V Foundation.

- IEEE Std 1076-2008. *IEEE Standard VHDL Language Reference Manual*.